Лекция 4 Эксперименты и базовая оценка модели

Методические материалы курса · учебный год 2026–2027

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа
Связанная работа: ЛР4 «Эксперименты и базовая оценка модели»

Цель лекции

Показать, как превратить обучение модели из единичного запуска в проверяемый эксперимент. После лекции студент должен уметь сформулировать гипотезу, определить изменяемый фактор, выбрать validation-метрики, зарегистрировать run в MLflow и обосновать решение без использования test-периода.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	От запуска к эксперименту	Сравнить «получили score» и проверяемую гипотезу
10–25 мин	Структура experiment и run	Разобрать parameters, metrics, tags, inputs, artifacts
25–42 мин	Базовая линия и сравнимость	Построить baseline и изменить один фактор
42–60 мин	Метрики бинарной классификации	Обсудить ROC AUC, precision, recall, F1 и порог
60–72 мин	Validation и закрытый test	Показать ошибку выбора по test
72–82 мин	Прослеживаемость MLflow	Связать run с Git и DVC
82–90 мин	Инструктаж к ЛР4	Уточнить два run и индивидуальную защиту

1 Почему запуск ещё не является экспериментом

Начните с вопроса: «Студент запустил обучение и получил ROC AUC 0,81. Что мы знаем?» Без версии данных, границ частей, параметров, baseline и контекста метрики почти ничего. Эксперимент начинается с проверяемой гипотезы и заканчивается воспроизводимым решением.

Используйте структуру:

1. Вопрос — какое изменение проверяется?
2. Гипотеза — почему оно должно повлиять на результат?
3. Контроль — что остаётся неизменным?
4. Метрика — по какому наблюдаемому сигналу принимается решение?
5. Ограничение — что мешает обобщить вывод?

Пример: увеличение max_depth может улучшить способность модели описывать нелинейные зависимости, но повысить риск переобучения. Данные, split и random seed остаются неизменными; сравнение проводится по validation ROC AUC и F1.

2 Базовая линия

Baseline — не самая простая модель вообще, а заранее определённая точка сравнения. Для задачи задержки рейса это может быть постоянное предсказание частоты положительного класса, логистическая регрессия или Random Forest с фиксированными параметрами.

Объясните две функции baseline:

- обнаружить, что сложная модель не превосходит тривиальный подход;
- обеспечить стабильную точку для регрессионных проверок.

Результат без baseline склонен к риторике «метрика выглядит высокой». Инженерное решение требует относительного сравнения и контекста стоимости ошибок.

3 Структура MLflow Tracking

Experiment группирует связанные runs. Run представляет одно выполнение кода и содержит:

Элемент	Пример	Назначение
Parameter	max_depth=4	входная настройка, не меняющаяся внутри run
Metric	validation_roc_auc=0.81	измеряемый результат
Tag	git.commit, validation.status	поиск, контекст и решение
Dataset input	training или validation	связь запуска с набором и digest
Artifact	модель, отчёт, график, manifest	воспроизводимый выход

Подчеркните отличие parameter и metric. Количество деревьев — параметр, время обучения или ROC AUC — метрика. Git commit не следует записывать как числовую метрику; для него подходит tag.

MLflow не делает эксперимент корректным автоматически. Он сохраняет то, что логирует код. Если команда не зафиксировала split или версию данных, красивый интерфейс не устраняет пробел.

4 Сравнимость двух runs

Для учебного эксперимента изменяйте один фактор. Если одновременно заменить данные, алгоритм и порог, невозможно объяснить источник изменения результата. В реальном исследовании применяются многофакторные планы, но они требуют достаточного числа запусков и анализа взаимодействий.

Перед сравнением проверьте:

- одинаковый Git commit или документированное отличие;
- одинаковый DVC hash и контракт;
- одинаковые временные границы;
- одинаковую реализацию метрик;
- одинаковый random seed для изоляции параметра;
- успешный статус и полный набор artifacts.

Если один run выполнен на другом периоде, сравнение метрик отражает одновременно модель и данные. Такой результат можно анализировать, но нельзя называть чистой проверкой гиперпараметра.

5 ROC AUC

ROC AUC оценивает ранжирование положительных и отрицательных примеров по score во всём диапазоне порогов. Значение 0,5 соответствует случайному ранжированию в стандартной бинарной постановке, 1,0 — идеальному.

Не называйте ROC AUC «процентом правильных ответов». Метрика не фиксирует рабочий порог и может выглядеть приемлемо при неудобном качестве редкого положительного класса. На validation с одним классом ROC AUC не определён; pipeline должен завершиться понятной ошибкой.

6 Precision, recall и F1

После выбора порога появляются бинарные решения:

- precision показывает долю истинных задержек среди предупреждений;
- recall показывает долю найденных задержек среди всех фактических задержек;

- F1 является гармоническим средним precision и recall.

Обсудите стоимость ошибок. Ложное предупреждение может вызвать лишнюю операционную реакцию, пропущенная задержка — сорвать планирование. Без бизнес-контекста нельзя объявить precision важнее recall или наоборот.

Порог 0,5 — исходная точка, а не естественный закон. В ЛР4 порог фиксируется для сравнимости. Его оптимизация должна проводиться только на validation и быть отдельным экспериментом.

7 Почему accuracy недостаточно

Если задерживаются 10 процентов рейсов, модель «всегда вовремя» имеет accuracy 90 процентов, но нулевой recall положительного класса. Покажите confusion matrix и попросите студентов вычислить смысл четырёх ячеек.

Accuracy можно логировать как вспомогательную метрику, но решение должно учитывать дисбаланс, стоимость ошибок и метрики положительного класса.

8 Train, validation и test

Train используется для fit модели и обучаемых преобразований. Validation служит для выбора алгоритма, гиперпараметров и порога. Test открывается после завершения выбора и оценивает уже зафиксированное решение.

Каждый повторный взгляд на test и последующая корректировка превращают test в ещё один validation. Оценка становится оптимистичной. В ЛР4 test-период существует в split manifest, но его метрики не используются для выбора run.

Отдельно объясните, что «мы только посмотрели» уже передаёт информацию. Информационная утечка может происходить через человеческое решение, а не только через строку кода.

9 Dataset inputs и безопасность

MLflow может зарегистрировать metadata и digest набора. Это полезно для поиска и сравнения. Но нельзя бездумно загружать закрытый CSV как artifact. Политика данных, лицензия и минимизация имеют приоритет.

Для каждого input укажите context training или validation, имя, источник, схему и digest. Свяжите это с DVC hash из ЛР3: DVC обеспечивает получение объекта, MLflow фиксирует его использование в конкретном run.

10 Артефакт модели

Артефакт считается полезным, если его можно загрузить и выполнить предсказание. Файл без формата, версии признаков и smoke test не подтверждает воспроизводимость.

В учебном bundle сохраняются модель, порядок признаков, версия, параметры, границы split и validation-метрики. После run другой участник загружает artifact во временный каталог и повторяет предсказание.

11 Статусы и решения

Не смешивайте факт run и решение о продвижении. Успешно завершённый run может быть candidate или rejected. Неуспешный из-за плохих данных получает invalid, а не удаляется из истории.

ЛР4 не использует Model Registry и production stages. Управляемый выпуск будет в ЛР10. Сейчас цель — научиться сравнивать runs и сохранять доказательства.

12 Типичные ошибки

- два run отличаются данными и параметрами одновременно;
- метрика называется score без контекста;
- Git commit и DVC hash отсутствуют;
- test используется для выбора;
- модель сохранена, но не загружается;
- неуспешный run удалён вместо анализа;

- персональные или закрытые данные загружены как artifact;
- вывод «модель лучше» сделан по малой разнице на крошечной выборке.

13 Индивидуальная ответственность

Каждый участник должен уметь открыть назначенный run, найти параметры, tags, inputs и artifacts, объяснить одну метрику и воспроизвести загрузку модели. Общий скриншот MLflow не заменяет личный артефакт.

Для команды из пяти человек распределите experiment ownership, модель, оценку, воспроизводимость и quality review. Роли должны ротироваться относительно ЛР3.

14 Вопросы аудитории

1. Чем parameter отличается от metric и tag?
2. Почему два run на разных данных нельзя напрямую сравнивать?
3. Что означает ROC AUC и чего он не показывает?
4. Когда precision важнее recall?
5. Почему accuracy может вводить в заблуждение?
6. Почему test нельзя использовать для настройки порога?
7. Что должен содержать загружаемый модельный artifact?
8. Зачем хранить неуспешные runs?

15 Завершение

Сформулируйте итог: MLflow превращает запуск в журнал доказательств, но научная и инженерная корректность определяется дизайном эксперимента. Два сравнимых runs должны иметь общие данные и split, различимый фактор, корректные validation-метрики и воспроизводимый artifact. ЛР5 проверит эти функции систематическими тестами.

Чек-лист преподавателя

- Показана структура hypothesis, control, metric и decision.
- Разобраны experiment, run, parameters, metrics, tags, inputs и artifacts.
- ROC AUC не интерпретируется как accuracy.
- Precision, recall и F1 связаны со стоимостью ошибок.
- Test остаётся закрытым при выборе.
- Run связан с Git commit и DVC hash.
- Индивидуальная защита требует воспроизведения, а не скриншота.

Официальные источники

- MLflow Tracking: https://mlflow.org/docs/latest/ml/tracking/
- MLflow Dataset Tracking: https://mlflow.org/docs/latest/dataset/
- scikit-learn metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html